

Jasper™ Superlint – LINT Overview

Version 2022.09

cadence®

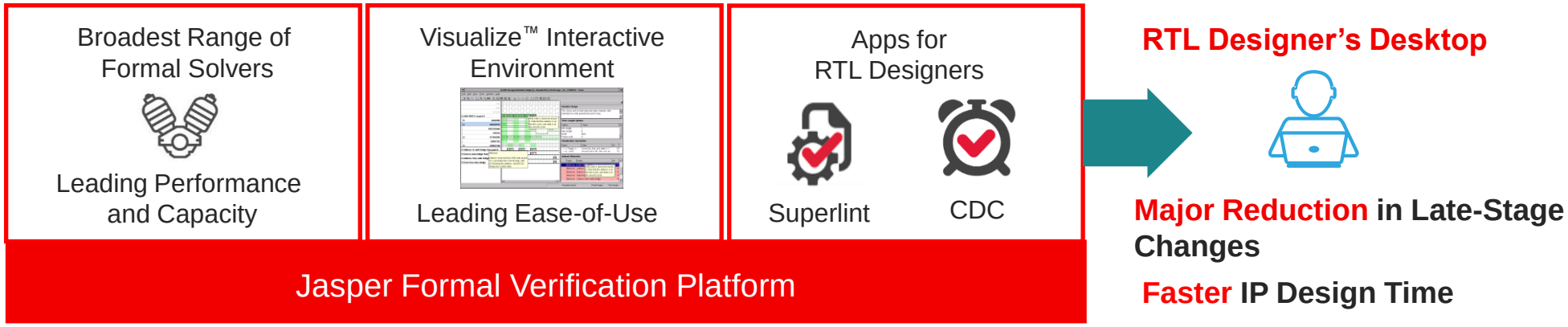
Agenda

- Introduction to Jasper Superlint
- Superlint - LINT Domains
- Superlint - LINT Flow
  - Configure Rules
  - Analyze and Elaborate RTL
  - Run Checks
  - Analysis and Debug
  - Create Waivers
  - Generate Reports
- Summary
- Jasper Superlint – LINT RAK Lab

© 2022 Cadence Design Systems, Inc. All Rights Reserved.

Jasper for RTL Signoff

- Jasper Superlint and Clock Domain Crossing (CDC) Apps reduce the IP development time of the logic designer by up to four weeks
  - Checks previously performed on the netlist are now moving to the RTL design stage
  - World-class formal technology now brought to the desktop of the RTL designer



© 2022 Cadence Design Systems, Inc. All Rights Reserved.

Comprehensive RTL Signoff – Big Picture

- Regular RTL Signoff Includes
  - Signoff the RTL against a comprehensive set of Structural LINT/DFT/CDC/RDC checks
  - All checks are ‘automatic’ and user just inputs RTL + Constraints (for DFT and CDC/RDC)

- Structural Checks should be augmented with Formal Assisted Automatic Checks
  - High-Value Auto-Formal Checks
    - Coverage: Dead code, FSM state reachability, Signal toggle, and more…
    - Functional checks: FSM Livelock/Deadlock, Bus contention, Arithmetic overflow, and more…
  - Auto-X Checks to check for unwanted X sources
  - Functional CDC/RDC checks and Metastability-aware DV (both Formal and Simulation)

© 2022 Cadence Design Systems, Inc. All Rights Reserved.

Superlint Domains

- Superlint comprises three domains:
  1. LINT
  2. DFT
  3. Auto-Formal
- Focus of this RAK is **LINT Domain**
  - Details of DFT and Auto-Formal are covered in separate RAKs

© 2022 Cadence Design Systems, Inc. All Rights Reserved.



Introduction to Superlint

© 2022 Cadence Design Systems, Inc. All Rights Reserved.

Superlint Overview

Structural LINT and DFT Checks

+

Automatic Formal Checks

DFT controllability

Sim-synth mismatch

Naming

Coding style

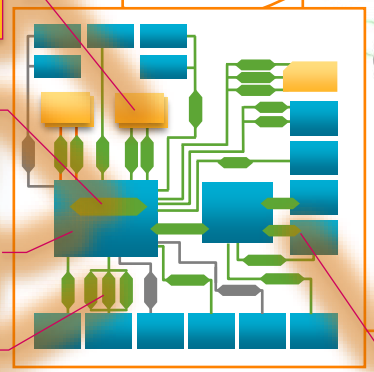
DFT observability

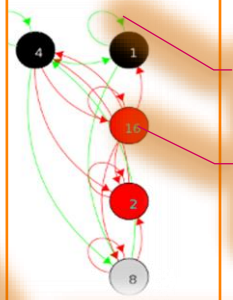
X assignment

Arithmetic overflow

Range overflow

Combo loop analysis





Reachability

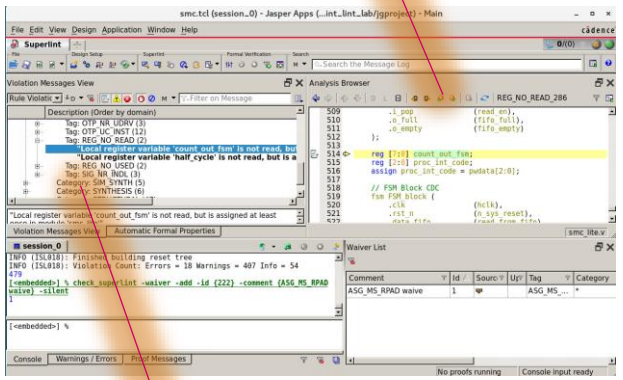
Livelock/ deadlock

Bus contention

Enabled by true formal technology

Best-in-class debug

Jasper Visualize™ Environment

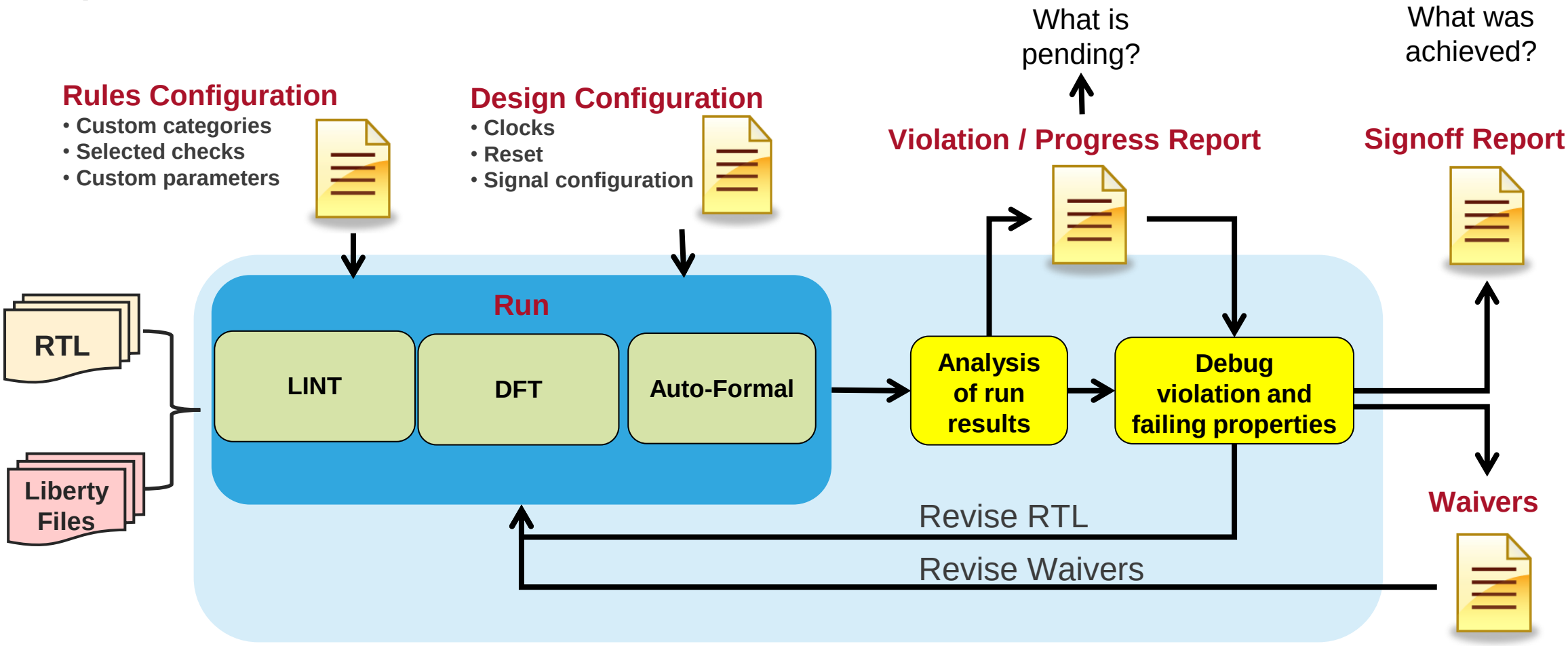


Low-noise violation and waiver handling

Comprehensive functional checks, violation debug, and waiver handling based on best-in-class formal analysis

© 2022 Cadence Design Systems, Inc. All Rights Reserved.

Superlint Flow



Note: Focus of this RAK is LINT. DFT and Auto-Formal are covered in separate RAKs.

© 2022 Cadence Design Systems, Inc. All Rights Reserved.

Superlint – LINT Domain

© 2022 Cadence Design Systems, Inc. All Rights Reserved.

LINT Domain Categories (1 of 3)

- NAMING
  - Checks naming conventions on RTL elements, for example modules, instances functions, etc. based on a rules file defined by user
- FILEFORMAT
  - Checks file formatting to make files portable and reusable, for example, file name different from module name
- CODINGSTYLE
  - Checks to ensure there are no semantic and functional issues in the code, for example, unconnected ports, unused and unassigned variables, or undriven signals
- SIM_SYNTH
  - Checks for scenarios that can cause mismatch between pre and post synthesis simulation results, for example, incomplete sensitivity list

© 2022 Cadence Design Systems, Inc. All Rights Reserved.

LINT Domain Categories (2 of 3)

- SYNTHESIS
  - Checks for constructs that are not synthesizable, for example, the Initial statement to initialize the signal in Verilog code can result in unpredictable synthesis behavior
- STRUCTURAL
  - Checks for design structures that can cause functional or downstream tool issues, for example, Flip-Flops missing resets
- RACES
  - Checks for simulation race condition scenarios. Race conditions occur when two events (with event order dependent code) take place at the same simulation time. For example, read and write on the same register in different always blocks

© 2022 Cadence Design Systems, Inc. All Rights Reserved.

LINT Domain Categories (3 of 3)

- CONNECTIVITY
  - Checks for the connectivity path provided with the `config_rtlds -connectivity` command. For example, mandatory connections between two nodes in the design

- BLACKBOX
  - Checks for blackbox related scenarios while elaborating the design. For example, explicit blackboxing of module/instance due to the `-bbox_m/-bbox_i` options passed to the `elaborate` command

© 2022 Cadence Design Systems, Inc. All Rights Reserved.

LINT Domain Categories – CODINGSTYLE Example

- Check name: `CAS_NR_DEFN` – Case statement incomplete (Severity Error)

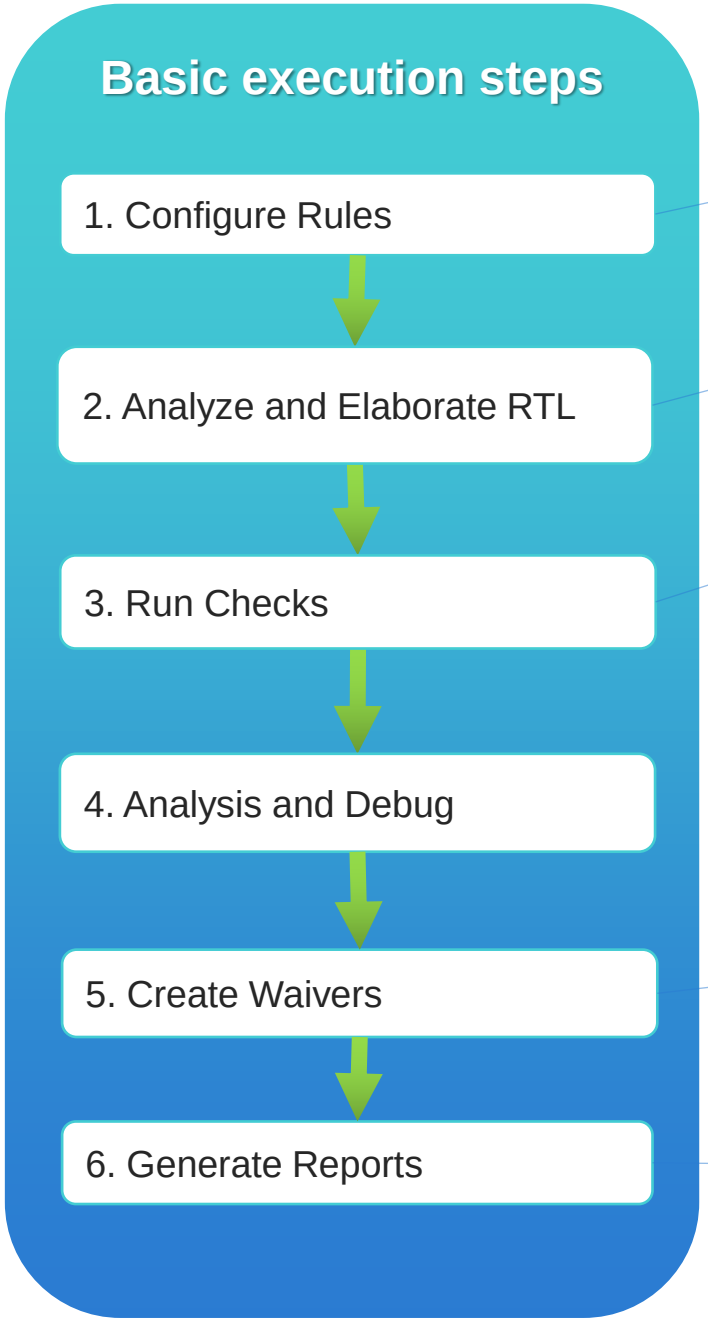
The following code illustrates the occurrence of `CAS_NR_DEFN`.

```
module CAS_NR_DEFN (a, b, c, sel, out1);
   input a;
   input b;
   input c;
   input [1:0] sel;
   output out1;
   reg out1;
   always @(a or b or c or sel)
   begin
      case (sel)
         2'b00: out1 = a;
         2'b01: out1 = b;
         2'b10: out1 = c;
      endcase
   end
endmodule
```

In the above code, all possible values of case selector are not specified. As a result, a violation is reported.

© 2022 Cadence Design Systems, Inc. All Rights Reserved.

Basic execution steps



config_rtlds -rule | ((-enable |-disable) (-tag |-category |-domain [all]) {<items_name_list>}) [-
severity (info | warning | error)] [-regexp]
config_rtlds -rule -load {<file_name_list>+}

analyze <other_options> <files_list>
elaborate -top <top>

check_superlint -extract [-instances <instance_list> |-hierarchies <instance_list>
|-exclude_hierarchies <instance_list> |-top] [-extended_property_name] [-task <task_name>] [-list]

check_superlint -waiver -add –l
(<message_id_tcl_list> | <property_id_tcl_list>) -comment <comment> [-silent] [-force]

check_superlint -report [-file <file_name> [-force] [(-html [-launch_html_browser])]]
[- violation [-severity (error | warning | info | fatal)] [-order (category | tag | filename |
instance(module) | label | severity)]] [-detailed]

© 2022 Cadence Design Systems, Inc. All Rights Reserved.

Superlint – LINT Flow

© 2022 Cadence Design Systems, Inc. All Rights Reserved.

Superlint LINT Flow

Rules Configuration
- Custom categories
- Selected checks
- Custom parameters

RTL

Liberty Files

Run

LINT | DFT | Auto-Formal

What is pending?

Violation / Progress Report

What was achieved?

Signoff Report

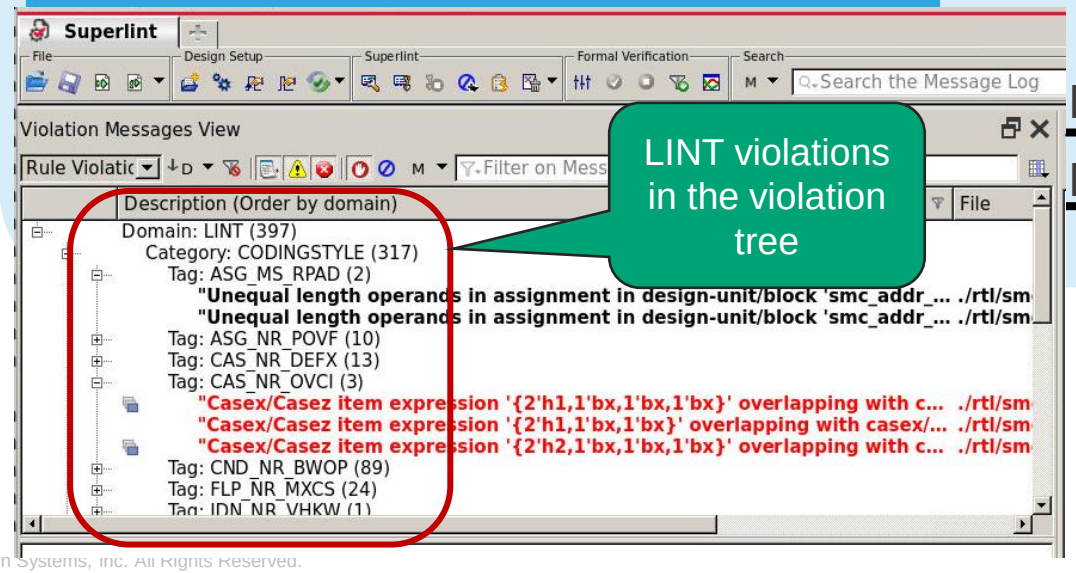
Analysis of run results

Debug violation and failing properties

Revise RTL

Revise Waivers

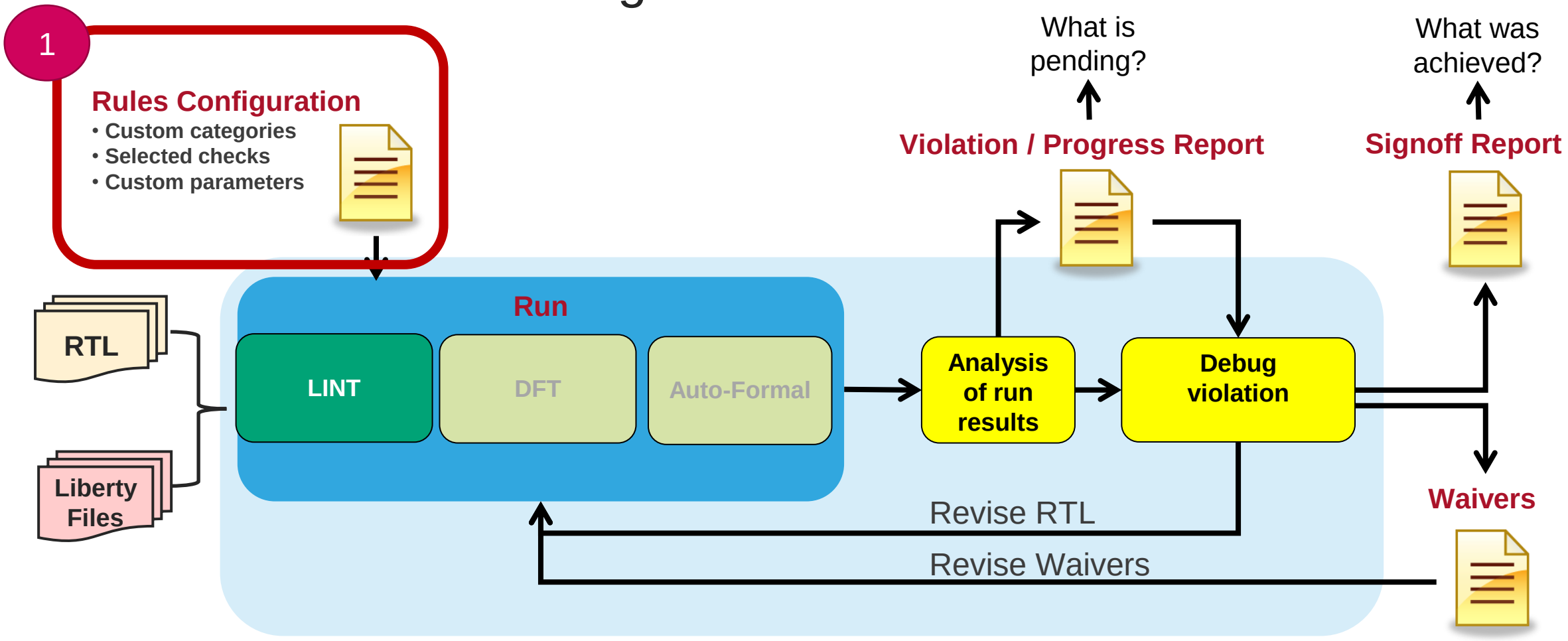
Waivers



LINT violations in the violation tree

© 2022 Cadence Design Systems, Inc. All Rights Reserved.

LINT Flow – Rules Configuration



© 2022 Cadence Design Systems, Inc. All Rights Reserved.

1.1. Configure Rules with Rules File

- Rules file contains the list of all LINT rules
  - The default rules file ‘superlint.def’, which contains the default Superlint categories and rules is located at:

    ```
    <jg_install> /etc/res/rtlds/rules/superlint.def
    ```

- You can create a custom rules file with the selected set of rules and customize the selected rules
- Customizations that you can do include:
  - Category names
    - Rules included in each category
    - Enable or disable a category
  - Severity
  - Short message
  - Rule name
  - Specific parameter for a rule, for example, pattern for naming convention rules

© 2022 Cadence Design Systems, Inc. All Rights Reserved.

1.2 Rules Configuration – Tcl Commands

- `config_rtlds -rule -load {file_name_list+}`: Load configuration from one or more ".def" files
- `config_rtlds -rule ((-enable | -disable) (-tag | -category | -domain) {items_name_list})`: To enable or disable the specified tags, categories, or domains
- `config_rtlds -rule ( -severity -tag tag_list |-parameter {key=value }+ -tag tag_list | -show (-tag {tag_list} |-category {category_list} |-domain {domain_list}))`: Manipulate default values of the parameters

© 2022 Cadence Design Systems, Inc. All Rights Reserved.

1.3 Rule Configuration – Examples of Rule Configuration

You can load the custom rules file using the following command:

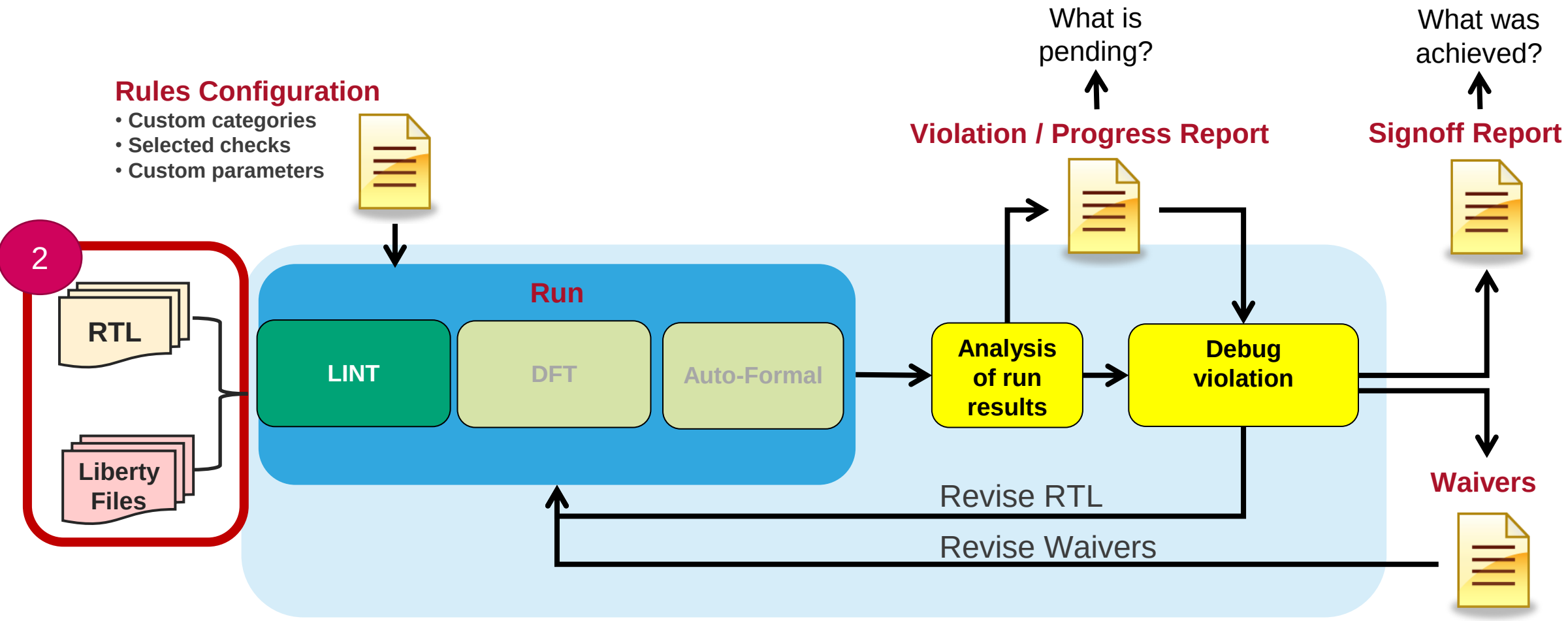
```
% config_rtlds -rule –load Superlint_Deployment_Rulefile_Lint_2022_09_Customer.def
```

To enable the LINT domain, use the following command:

```
% config_rtlds -rule -enable -domain {LINT}
```


© 2022 Cadence Design Systems, Inc. All Rights Reserved.

LINT Flow – Analyze and Elaborate RTL



© 2022 Cadence Design Systems, Inc. All Rights Reserved.

2. Analyze and Elaborate RTL

- Following commands are used to compile and elaborate:

```
% analyze <other_options> <files_list> : Analyze RTL files
% elaborate -top <top>                 : Elaborate
```

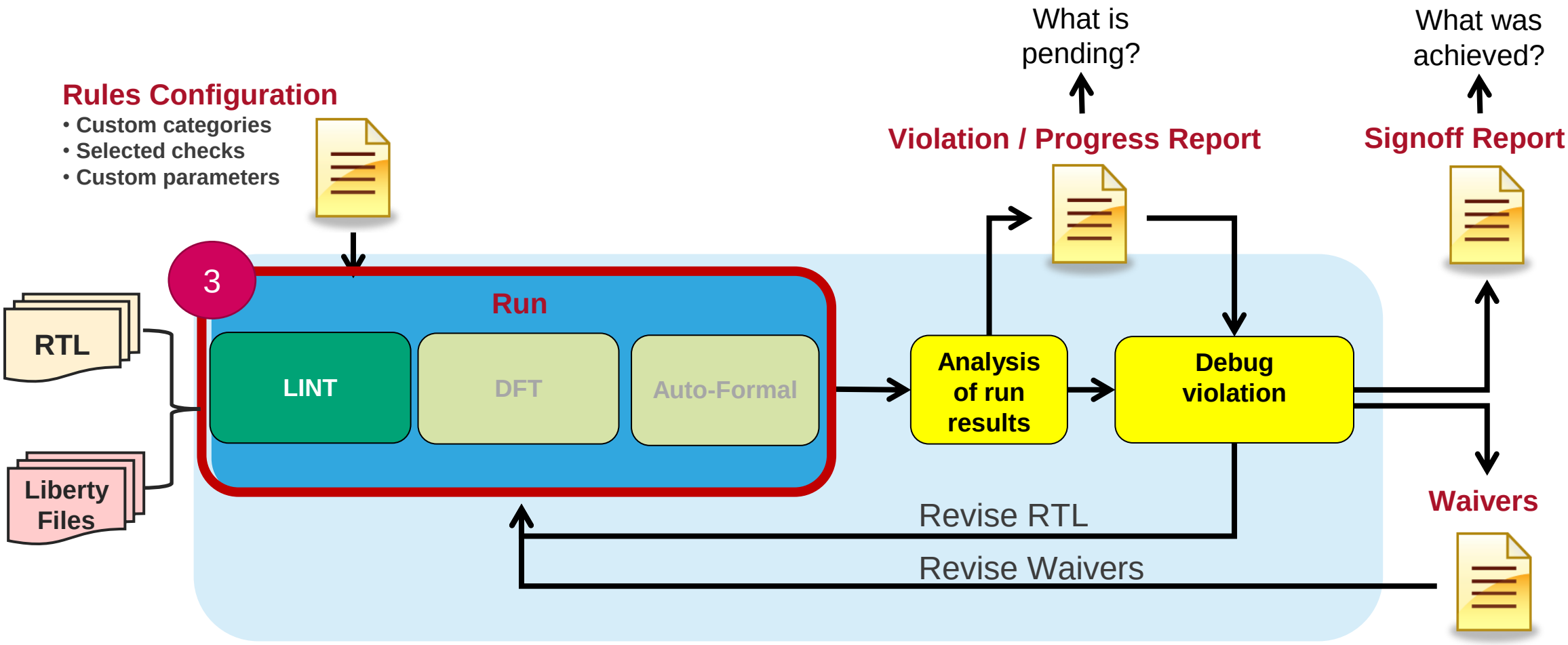
- The console log window shows analysis and elaboration messages for the DUT. There should be no error messages in the console log. Example:

```
% analyze -sv -f rtl/apb_subsystem.f
% elaborate -top smc_veneer
```

```
%
% analyze -sv -f rtl/apb_subsystem.f
INFO (INL011): Processing "-f" file "/net/noivl-vivek/export/home/vivek/customer/jasper/sl_rak/
10Sept/rak_jaspergold_superlint_batch_mode_lab/rak_jasper_superlint_lint_lab_database/rtl/
apb_subsystem.f".
% elaborate -top smc_veneer
INFO (INL208): Elaboration synthesis summary:
  Number of synthesized netlists          19 (0 packages)
  Single run mode                         On
  Pipeline                                On (19 pipelined netlists)
  Late hierarchical reference connection  On
  Number of analyzed Verilog modules      27 (19 synthesized)
  Number of analyzed VHDL entities        0 (0 synthesized)
INFO (INL003): Clearing all state information (assumes, stopats, and so forth).
smc veneer
```


© 2022 Cadence Design Systems, Inc. All Rights Reserved.

LINT Flow – Run Checks



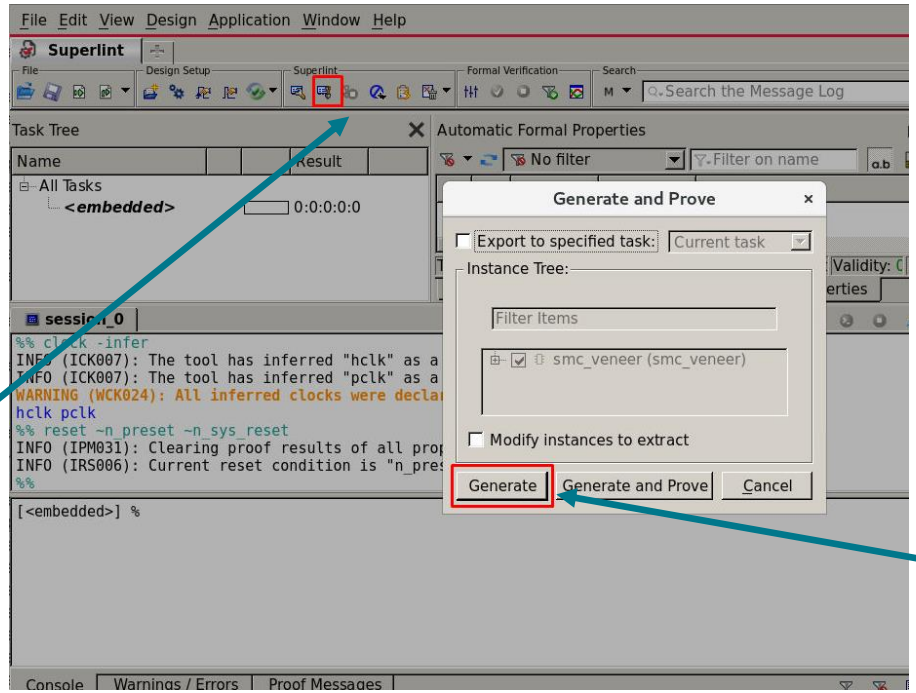
© 2022 Cadence Design Systems, Inc. All Rights Reserved.

3. Run Checks

- Following Tcl command can be used to run/generate the checks:

```
% check_superlint -extract
```

You can also use the GUI: Click the **Generate Superlint Checks** button on the **Superlint** toolbar and use the dialog to run the LINT checks

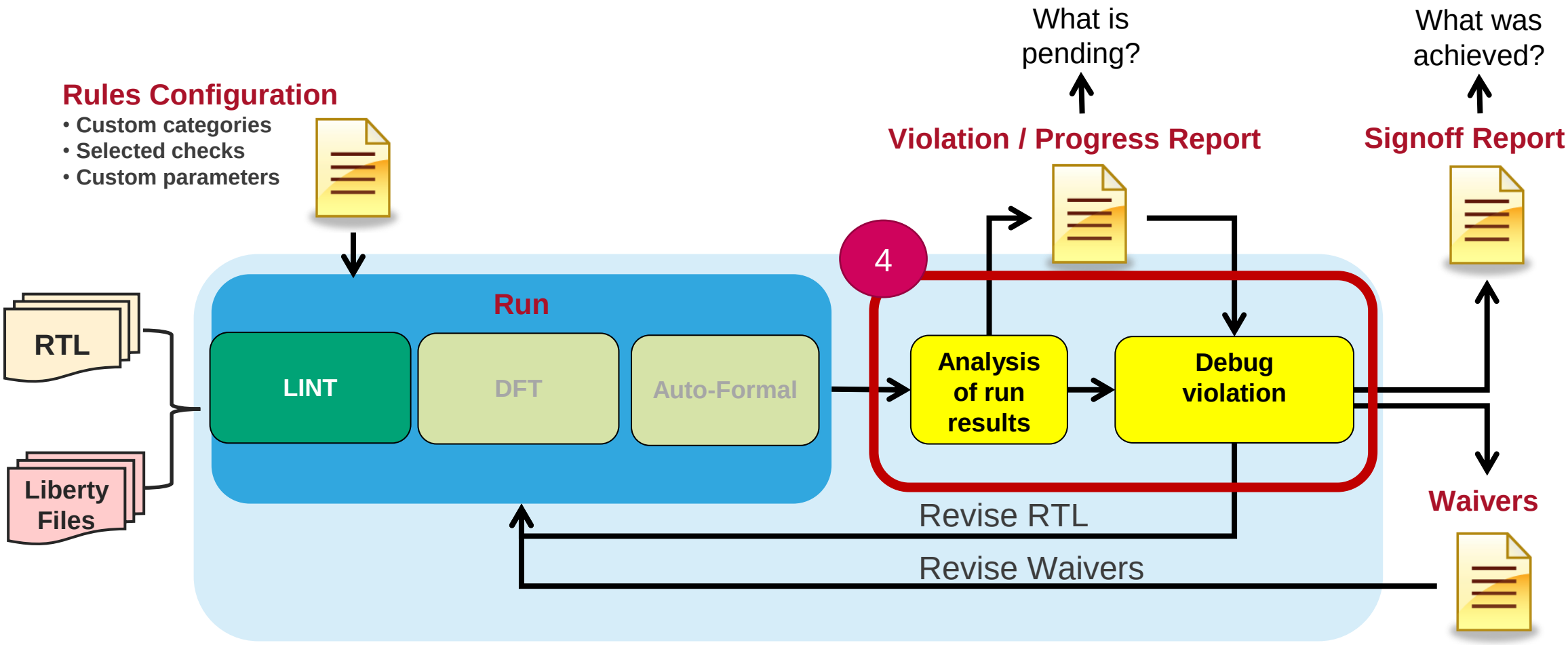


Generate Superlint checks

Click on Generate to extract LINT checks

© 2022 Cadence Design Systems, Inc. All Rights Reserved.

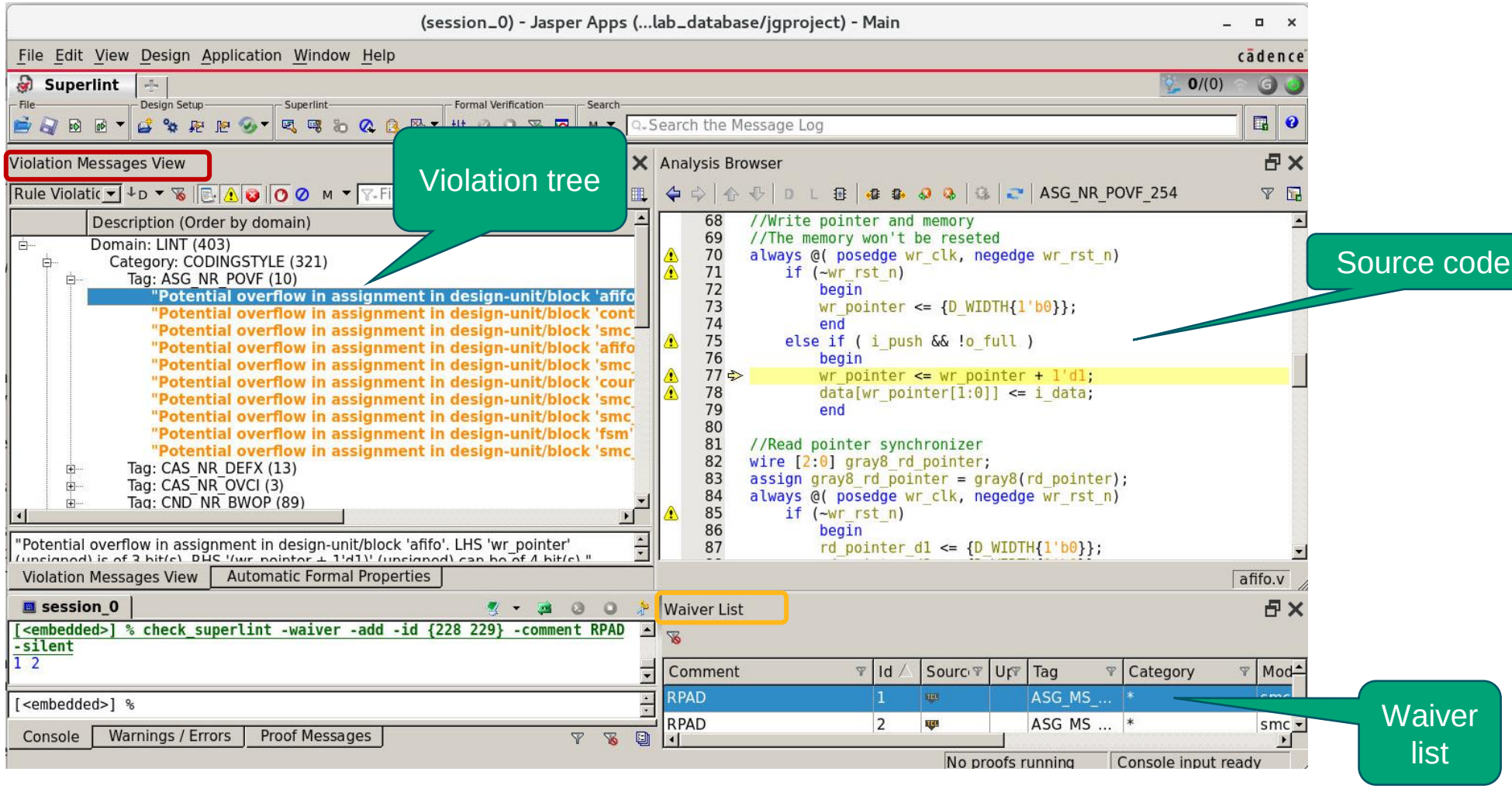
Superlint Flow – Analysis and Debug



© 2022 Cadence Design Systems, Inc. All Rights Reserved.

4.1 Analysis and Debug – Violation Layout

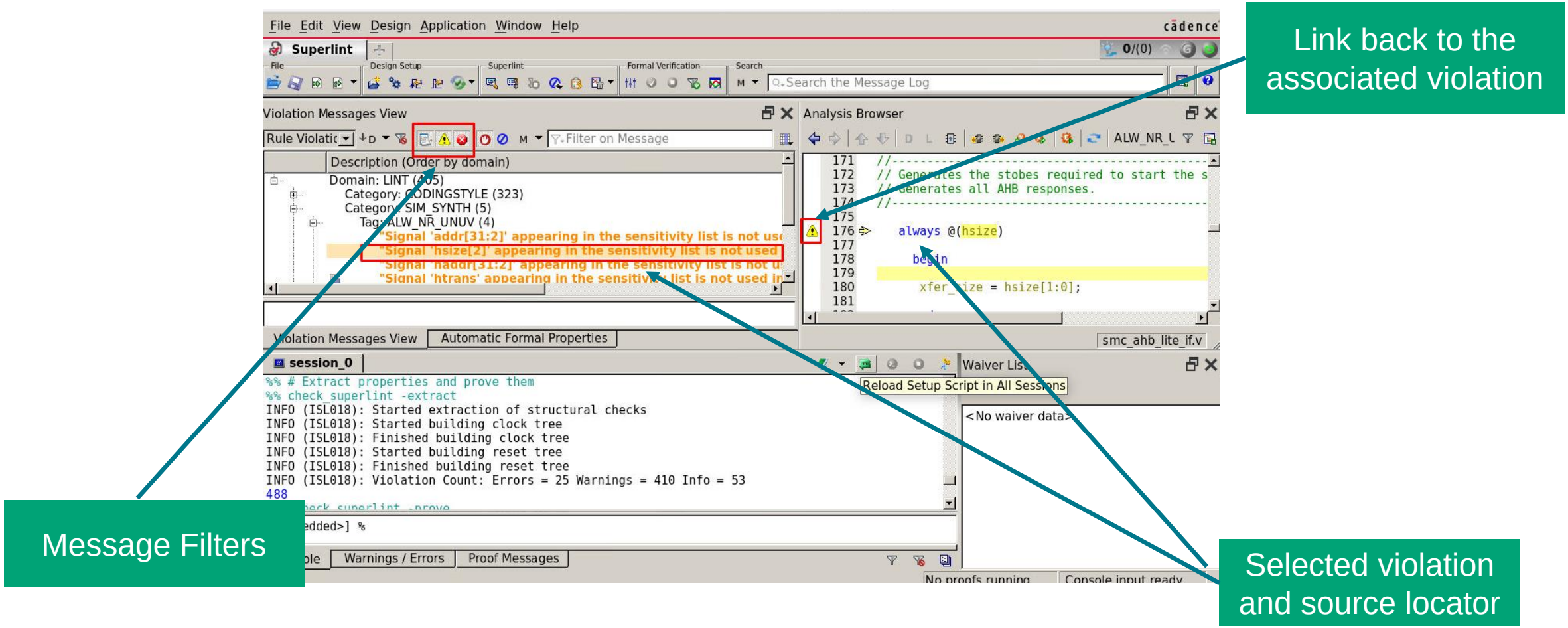
Expand the **Violation Messages View** tree. Violation tree, source code, and waiver list can be viewed in the layout.



© 2022 Cadence Design Systems, Inc. All Rights Reserved.

4.2 Analysis and Debug – Reviewing Violations

Expand the **Violation Messages View** tree and select any desired violation. The **Analysis Browser** locates the corresponding source code with an arrow symbol.

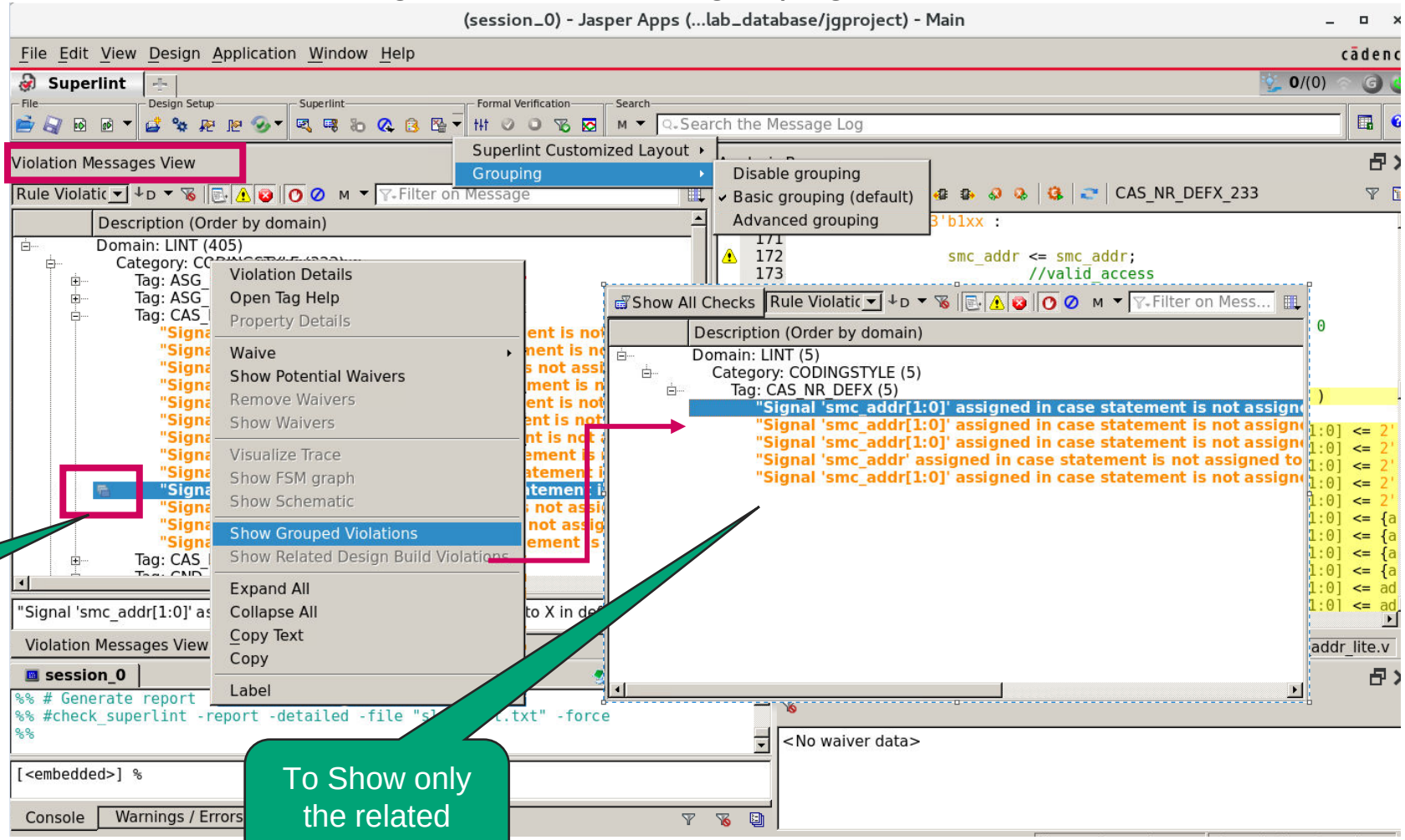


© 2022 Cadence Design Systems, Inc. All Rights Reserved.

4.3 Analysis and Debug – Grouping Feature

- The tool groups similar checks
  - Shows only the primary check in the violation message view and adds a grouping icon to the left of this check

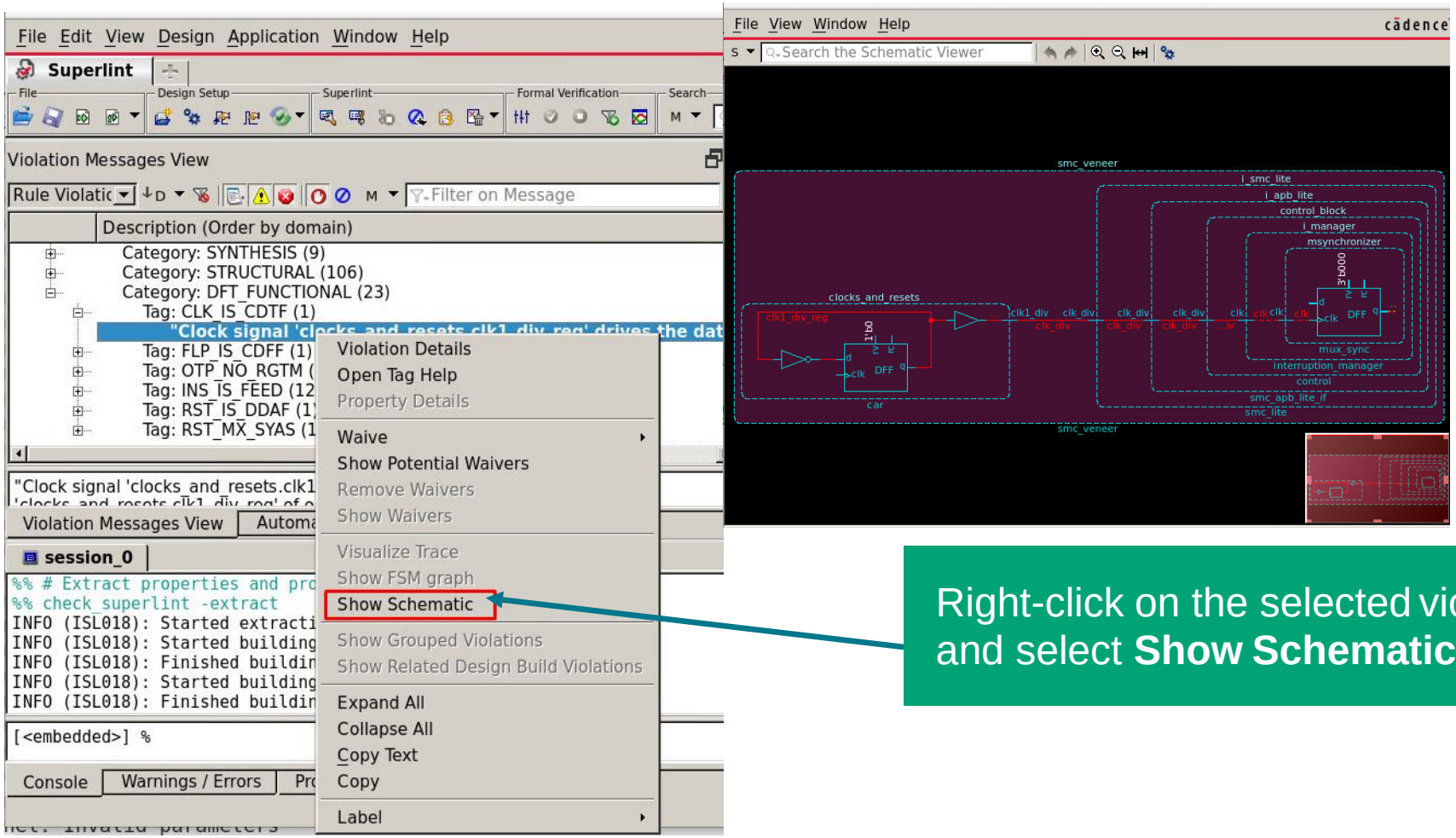
You can ungroup a particular primary violation to see the associated secondary violations.



© 2022 Cadence Design Systems, Inc. All Rights Reserved.

4.4 Analysis and Debug – Using Schematic

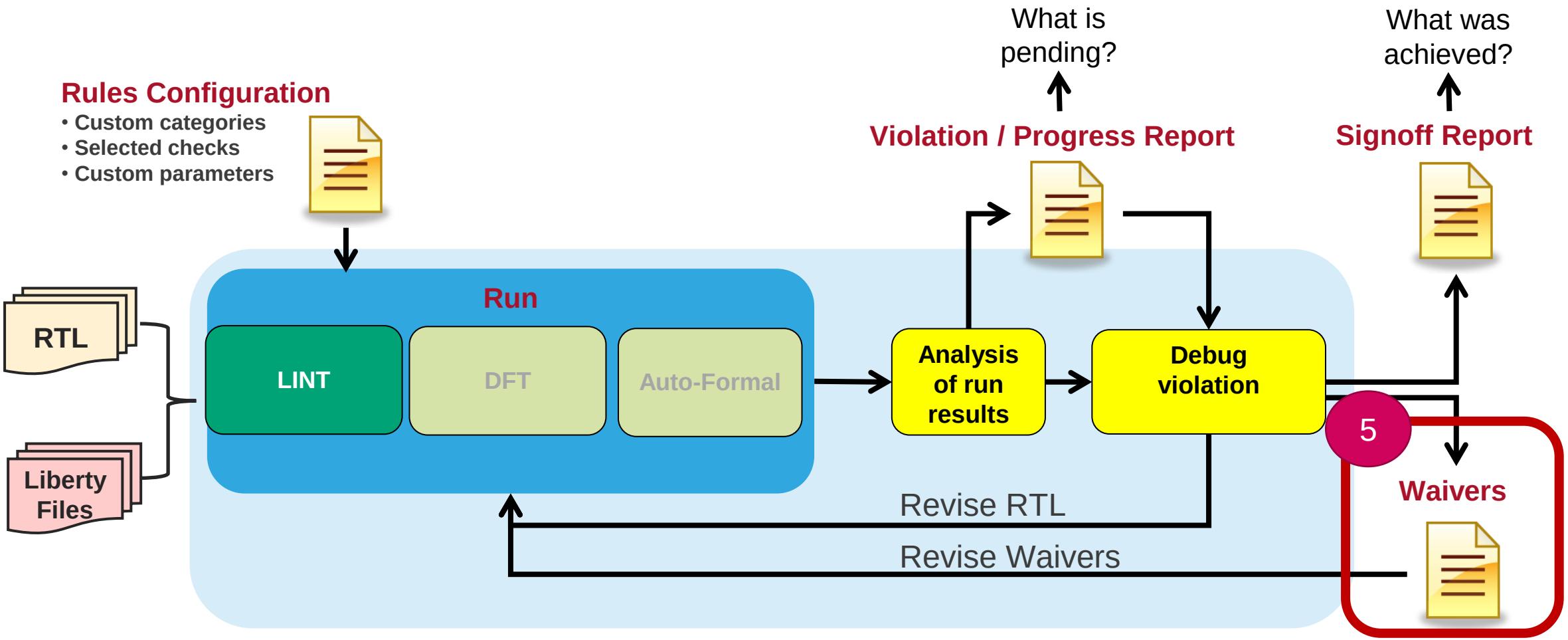
For structural violations, right-click and select **Show Schematic**.



Right-click on the selected violation and select **Show Schematic**

© 2022 Cadence Design Systems, Inc. All Rights Reserved.

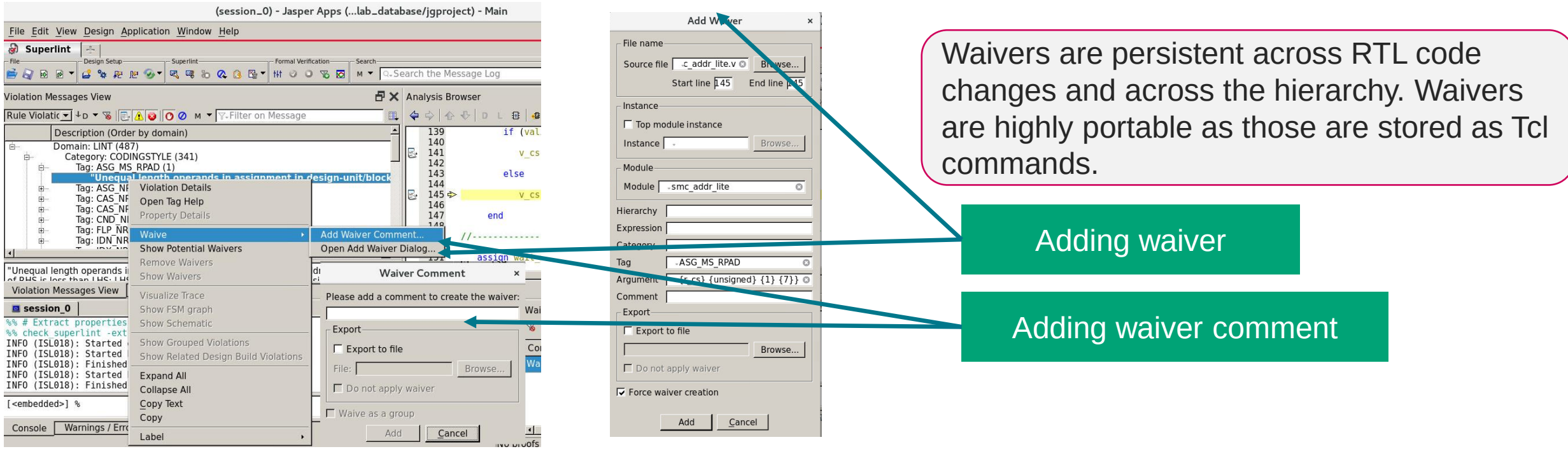
LINT Flow – Create Waivers



© 2022 Cadence Design Systems, Inc. All Rights Reserved.

5. Create Waivers

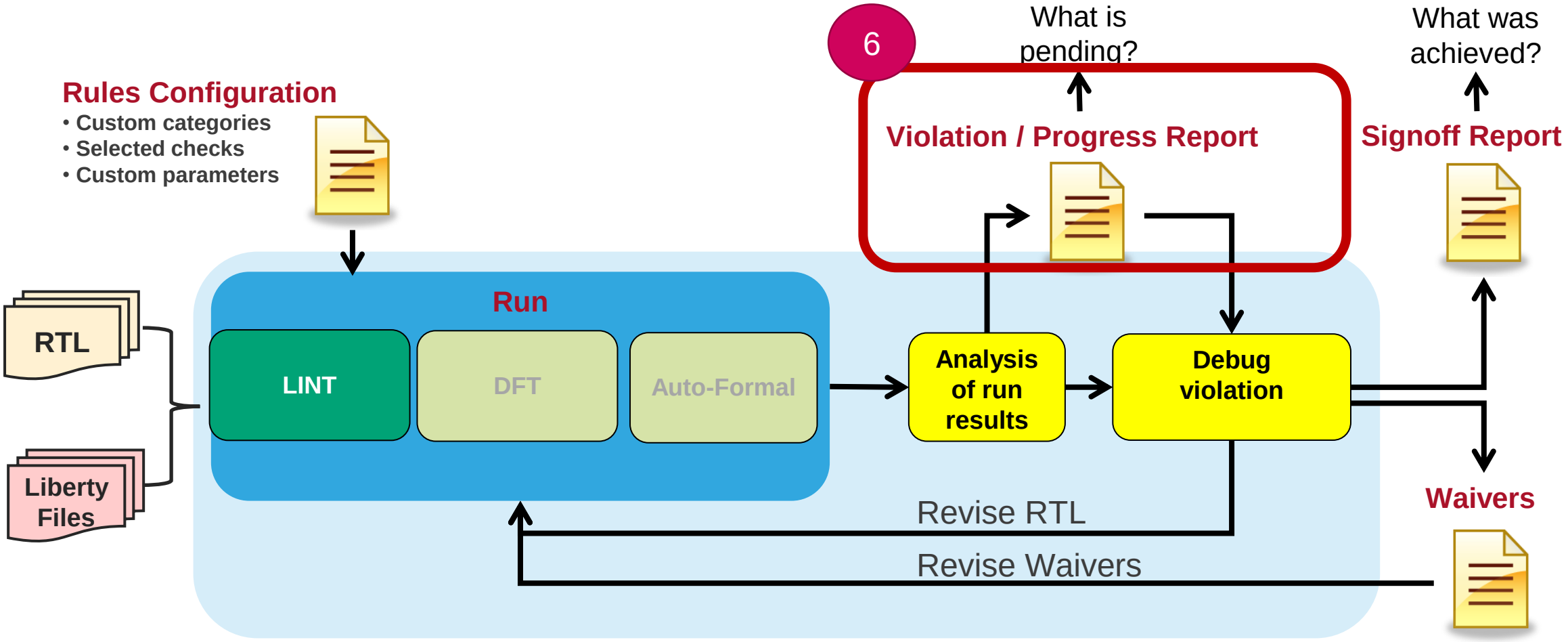
You can create waivers by selecting one or more violations in Violation Messages View. Also, you can waive violations with the **Add Waiver** dialog shown below to get additional waiver options. Waiver export and import options are also available.



Tcl Command: check_superlint -waiver -add –l
(<message_id_tcl_list> | <property_id_tcl_list>) -comment <comment> [-silent] [-force]

© 2022 Cadence Design Systems, Inc. All Rights Reserved.

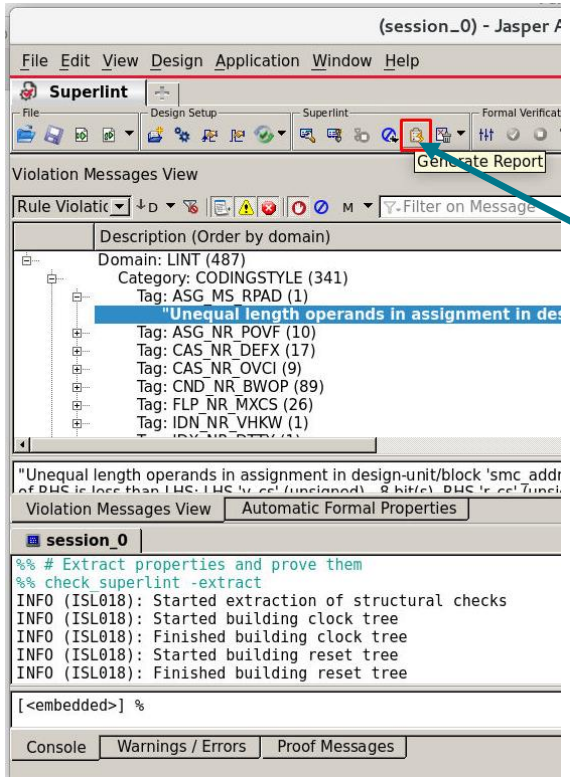
LINT Flow – Generate Reports

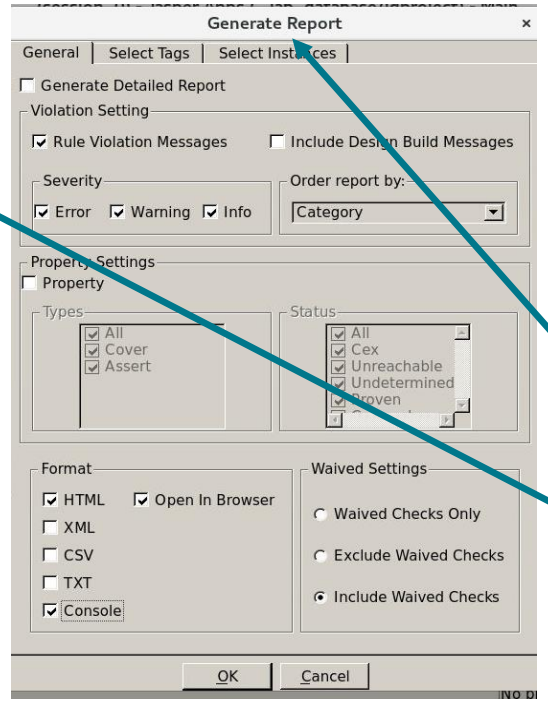


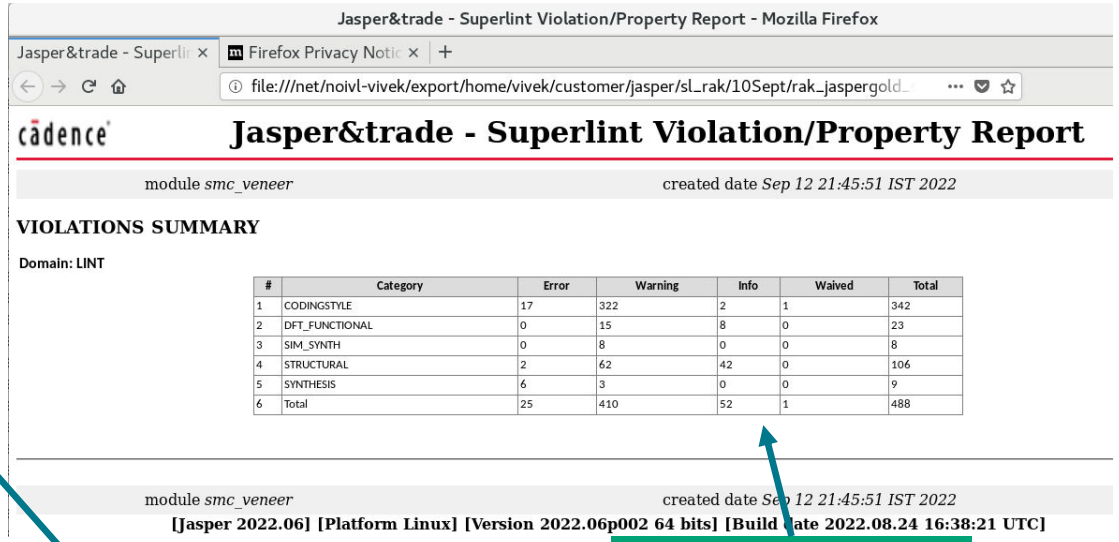
© 2022 Cadence Design Systems, Inc. All Rights Reserved.

6. Generate Reports

Generate reports by choosing the Generate Report button on the Superlint toolbar. The dialog below shows report generation options.







HTML Report Example

Generate Report

Tcl Command: check_superlint -report [-file <file_name> [-force] [(-html [-launch_html_browser])]] [- violation [-severity (error | warning | info | fatal)] [-order (category | tag | filename | instance(module) | label | severity)]] [-detailed]

© 2022 Cadence Design Systems, Inc. All Rights Reserved.

Summary

- Superlint provides exhaustive LINT checks for early detection of design bugs
- Enables more structured designer-level verification
- Is highly customizable
  - You can run with custom rules file and the selected subset of rules
- Provides advanced analysis and debug capabilities
  - Link back to the source
  - Schematic
  - Advanced waiver and report capabilities
- Is designed to be a low-noise and high-productivity application

© 2022 Cadence Design Systems, Inc. All Rights Reserved.



Jasper Superlint – LINT RAK Lab

© 2022 Cadence Design Systems, Inc. All Rights Reserved.

Introduction

In this RAK, you will learn how to run Superlint. Focus is on the following sections, and you will step through a simple example of each of the following procedures:

- Configure Checks
- Analyze and Elaborate RTL
- Run Checks
- Analyze and Debug
- Create Waivers
- Generate Reports

© 2022 Cadence Design Systems, Inc. All Rights Reserved.

Requirements for Running the RAK Lab Exercise

- Required product versions:
  - Jasper 2022.09 (or later)

  Note: This lab is specific to version 2022.09. If you are running a different version, you may see differences in the user interface and supported checks.

- Required licenses:
  - `jasper_pint + jasper_papp`

  OR

  - `jasper_pint + jasper_dao`

© 2022 Cadence Design Systems, Inc. All Rights Reserved.

Support and Feedback

- **Support**

  Cadence Learning and Support Portal provides access to support resources, including an extensive knowledge base, access to software updates for Cadence products, and the ability to interact with Cadence Customer Support. Visit https://support.cadence.com.

- **Feedback**

  Email comments, questions, and suggestions to content_feedback@cadence.com.

© 2022 Cadence Design Systems, Inc. All Rights Reserved.



© 2022 Cadence Design Systems, Inc. All rights reserved worldwide. Cadence, the Cadence logo, and the other Cadence marks found at www.cadence.com/go/trademarks are trademarks or registered trademarks of Cadence Design Systems, Inc. Accellera and SystemC are trademarks of Accellera Systems Initiative Inc. All Arm products are registered trademarks or trademarks of Arm Limited (or its subsidiaries) in the US and/or elsewhere. All MIPI specifications are registered trademarks or service marks owned by MIPI Alliance. All PCI-SIG specifications are registered trademarks or trademarks of PCI-SIG. All other trademarks are the property of their respective owners.